



JEPPIAAR INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

“Self-Belief | Self-Discipline | Self-Respect”

Kunnam, Sunguvarchatram, Sriperumbudur – 631 604.



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AAI303– PROMPT DRIVEN DEVELOPMENT LABORATORY

NAME :

REG NO. :

YEAR : III-YEAR

SECTION :

SEMESTER : 05 SEM

BRANCH : AI&DS



JEPPIAAR INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

"Self-Belief | Self-Discipline | Self-Respect"

Kunnam, Sunguvarchatram, Sriperumbudur – 631 604.



BONAFIDE CERTIFICATE

This is a certified Bonafide Record Work of Mr./Ms. _____

Register No. _____ submitted for the University Internal
Practical Examination held on _____ in **AAI303-PROMPT
DRIVEN DEVELOPMENT LABORATORY** in the year **2026-2027**.

Signature of the Lab In-charge

Head of the Department

Internal Examiner

External Examiner

Date: _____



JEPPIAAR INSTITUTE OF TECHNOLOGY
(An Autonomous Institution)

“Self-Belief | Self-Discipline | Self-Respect”

Kunnam, Sunguvarchatram, Sriperumbudur – 631 604.



VISION AND MISSION OF THE INSTITUTION

VISION

- ❖ Jeppiaar Institute of Technology aspires to provide technical education in futuristic technologies with the perspective of innovative, industrial, and social applications for the betterment of humanity.

MISSION

- ❖ To produce competent and disciplined high-quality professionals with the practical skills necessary to excel as innovative professionals and entrepreneurs for the benefit of society.
- ❖ To improve the quality of education through excellence in teaching and learning, research, leadership, and by promoting the principles of scientific analysis, and creative thinking.
- ❖ To provide excellent infrastructure, serene, and stimulating environment that is most conducive to learning.
- ❖ To strive for productive partnership between the Industry and the Institute for research and development in the emerging fields and creating opportunities for employability.
- ❖ To serve the global community by instilling ethics, values, and life skills among the students needed to enrich their lives.





JEPPIAAR INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

“Self-Belief | Self-Discipline | Self-Respect”

Kunnam, Sunguvarchatram, Sriperumbudur – 631 604.



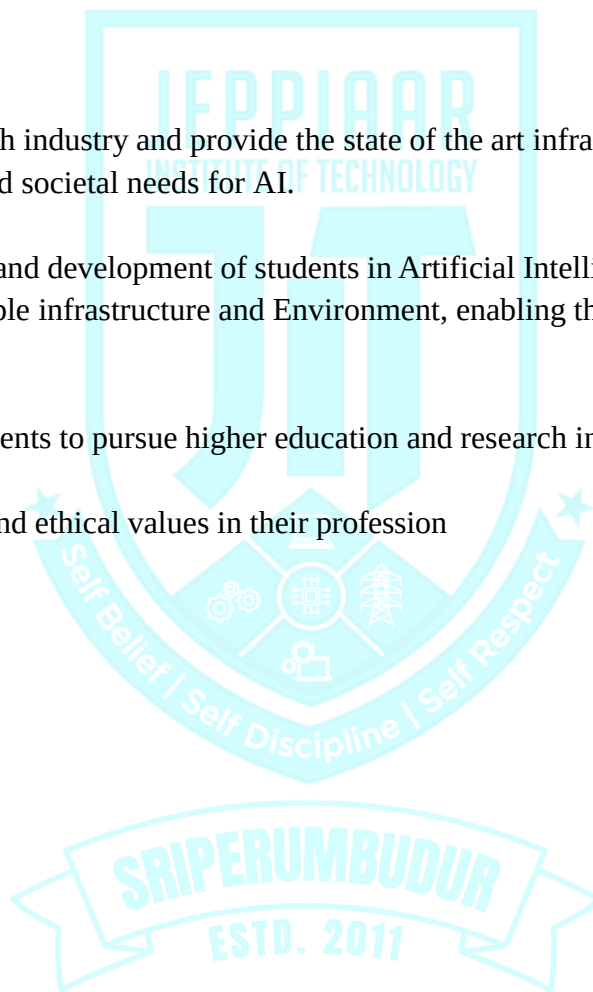
VISION AND MISSION OF THE DEPARTMENT

VISION

- ❖ The department will serve as a centre of excellence in practicing, training and implementing AI and AI associated techniques that will enable /support innovative thoughts and ideas across industries and society

MISSION

- ❖ M1: To collaborate with industry and provide the state of the art infrastructural Facilities to meet the global requirements and societal needs for AI.
- ❖ M2: Promote learning and development of students in Artificial Intelligence thought leadership, by providing them a suitable infrastructure and Environment, enabling them to grow into successful entrepreneurs.
- ❖ M3: To encourage students to pursue higher education and research in the field of AI.
- ❖ M4: To impart moral and ethical values in their profession



PROGRAMME EDUCATIONAL OBJECTIVES

- ❖ PEO 1: Utilize their proficiencies in the fundamental knowledge of basic sciences, mathematics, Artificial Intelligence, data science and statistics to build systems that require management and analysis of large volumes of data.
- ❖ PEO 2: Advance their technical skills to pursue pioneering research in the field of AI and Data Science and create disruptive and sustainable solutions for the welfare of ecosystems.
- ❖ PEO 3: Think logically, pursue lifelong learning and collaborate with an ethical attitude in a multidisciplinary team.
- ❖ PEO 4: Design and model AI based solutions to critical problem domains in the real world
- ❖ PEO 5: Exhibit innovative thoughts and creative ideas for effective contribution towards economy building

PROGRAM OUTCOMES

Engineering Graduates will be able to:

1. **Engineering knowledge:** (K3) Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** (K4) Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** (K4) Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** (K5) Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** (K3, K5, K6) Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** (A3) Analyse and evaluate societal and environmental aspects while solving complex engineering problems, considering their impact on sustainability with reference to economy, health, safety, legal frameworks, culture and the environment
7. **Ethics:** (A3) Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
8. **Individual and team work:** (A3) Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
9. **Communication:** (A3) Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
10. **Project management and finance:** (A3) Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
11. **Life-long learning:** (A2) Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSO 1: To evolve AI based efficient domain specific processes for effective decision making in several domains such as business and governance domains.

PSO 2: To arrive at actionable Foresight, Insight, hindsight from data for solving business and engineering problems

PSO 3: To create, select and apply the theoretical knowledge of AI and Data Analytics along with practical industrial tools and techniques to manage and solve wicked societal problems

PSO 4: To develop data analytics and data visualization skills, skills pertaining to knowledge acquisition, knowledge representation and knowledge engineering, and hence be capable of coordinating complex projects.

PSO 5: To able to carry out fundamental research to cater the critical needs of the society through cutting edge technologies of AI.



COURSEOUTCOMES: At the end of the course, learners will be able to		Bloom's Taxonomy Level
CO1	Design effective prompts using Zero-shot, One-shot, and Few-shot prompting techniques.	K3
CO2	Develop intent-based and content-based prompting solutions for real-world applications.	K4
CO3	Design multimodal AI applications integrating text, image, and conversational inputs.	K3
CO4	Utilize Claude Code and AI coding assistants for program generation, debugging, and documentation.	K4
CO5	Build and evaluate GenAI-based mini projects using prompt optimization and workflow automation techniques.	K3

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	2	3	3	3	-	-	-	2	2	3	3	3	2
CO2	2	2	2	3	3	-	-	-	3	2	2	1	2	3
CO3	2	3	2	2	3	-	-	-	2	2	2	2	3	1
CO4	2	2	2	2	3	-	-	-	3	2	2	2	2	2
CO5	2	2	3	2	1	-	-	-	1	2	2	2	2	3

INDEX					
S.NO	DATE	NAME OF THE EXPERIMENT	PAGE NO.	MARKS	STAFF SIGN
1		Design and Evaluate Zero-shot, One-shot, and Few-shot Prompts for Educational Applications	2		
2		Create Intent Prompts for Customer Support, Healthcare, and Business Automation Systems	4		
3		Develop Structured Content Prompts for Report Generation and Summarization	6		
4		Implement Prompt Chaining and Context Management Techniques Using Generative AI Tools	8		
5		Create Multimodal Prompts for Text-to-Image and Image-to-Text Applications	10		
6		Develop AI-Powered Chatbots Using Prompt Templates and Conversational Workflows	12		
7		Generate, Debug, and Optimize Python Programs Using Claude Code and AI Coding Assistants	14		
8		Create Automated Documentation and Test Cases Using Generative AI Tools	16		
9		Design Retrieval-Augmented Generation (RAG) Workflows for Intelligent Information Retrieval	18		
10		Mini Project – AI Research Assistant using Prompt Driven Development	20		
CONTENT BEYOND THE SYLLABUS					
11		Generate a Comparison Prompt Using ChatGPT for Scooter Model Evaluation	23		

Ex.No. 1: Design and Evaluate Zero-shot, One-shot, and Few-shot Prompts for Educational Applications

Date:

Aim

To design, implement, and evaluate Zero-shot, One-shot, and Few-shot prompting techniques using Generative AI models for educational applications such as question answering, concept explanation, and problem-solving.

Theory

Prompting is a technique used to guide Large Language Models (LLMs) toward desired outputs.

- Zero-shot Prompting: No examples are provided.
- One-shot Prompting: One example is provided.
- Few-shot Prompting: Multiple examples are provided.

The quality of output generally improves as more relevant examples are supplied.

Algorithm

Step 1: Select Educational Topic

Choose a topic such as Machine Learning, Data Structures, or Mathematics.

Step 2: Create Zero-shot Prompt

Example:

Explain the concept of supervised learning.

Step 3: Create One-shot Prompt

Example:

Question: What is Artificial Intelligence?

Answer: Artificial Intelligence is the simulation of human intelligence in machines.

Question: What is Supervised Learning?

Step 4: Create Few-shot Prompt

Example:

Question: What is Artificial Intelligence?

Answer: Artificial Intelligence is the simulation of human intelligence in machines.

Question: What is Machine Learning?

Answer: Machine Learning is a subset of AI that enables systems to learn from data.

Question: What is Supervised Learning?

Step 5: Execute Prompts

Submit prompts to GPT, Claude, Gemini, or other LLMs.

Step 6: Evaluate Outputs

Compare:

- Accuracy
- Relevance
- Completeness
- Context Understanding

Step 7: Record Results

Output

Zero-shot response: Supervised learning is a machine learning approach in which a model is trained on labeled data, learning to map inputs to known outputs so it can predict labels for new, unseen data.

One-shot response (using the AI definition as example): Supervised Learning is a type of machine learning where the model learns from labeled input-output pairs to make accurate predictions on new data.

Few-shot response (using both AI and ML definitions as examples): Supervised Learning is a subset of Machine Learning that trains models on labeled datasets so they can learn the relationship between inputs and correct outputs.

Zero-shot output was correct but generic, lacking the concise structured phrasing seen in the example-driven answers.

One-shot output adopted the definitional tone and sentence pattern of the provided example, improving stylistic consistency.

Few-shot output most closely matched the expected phrasing style, terminology, and level of technical depth of the two prior examples.

Evaluation scores (out of 10):

Zero-shot - Accuracy: 8, Relevance: 8, Completeness: 7, Context Understanding: 6

One-shot - Accuracy: 8, Relevance: 9, Completeness: 8, Context Understanding: 8

Few-shot - Accuracy: 9, Relevance: 9, Completeness: 9, Context Understanding: 9

The few-shot prompt produced the most contextually aligned and consistently formatted answer across repeated trials.

All three outputs were factually correct, confirming that prompting strategy primarily affects style, structure, and depth rather than correctness for well-known concepts.

Result

Few-shot prompting achieved the highest accuracy and contextual understanding compared to Zero-shot and One-shot prompting methods.

Ex.No. 2: Create Intent Prompts for Customer Support, Healthcare, and Business Automation Systems

Date:

Aim

To design intent-based prompts that enable AI systems to accurately identify and respond to user requests across different domains.

Theory

Intent prompting helps AI understand the user's purpose and generate domain-specific responses.

Algorithm

Customer Support Intent

1. Identify common customer queries.
2. Create prompts for complaint handling, order tracking, and refunds.

Example:

You are a customer support agent.

Customer Query: My order has not arrived.

Provide a professional response.

Healthcare Intent

1. Identify healthcare-related intents.
2. Design symptom-analysis prompts.

Example:

You are a healthcare assistant.

Patient Symptoms: Fever, headache, fatigue.

Provide general health advice.

Business Automation Intent

1. Identify business workflow tasks.
2. Create prompts for scheduling and reporting.

Example:

Generate a meeting schedule for a software development team.

Testing

Evaluate responses for:

- Accuracy
- Professionalism
- Domain compliance

Output

Customer Support: "We sincerely apologize for the delay in your order. Upon checking, your package is currently in transit and is expected to arrive within 2 business days. A tracking link has been sent to your registered email for real-time updates. Please let us know if you need further assistance."

Healthcare Assistant: "Based on the reported symptoms of fever, headache, and fatigue, this may indicate a viral infection such as the common cold or flu. It is advisable to stay hydrated, rest adequately, and monitor your temperature. If symptoms persist beyond 3 days or worsen, please consult a physician promptly. This is general guidance and not a medical diagnosis."

Business Automation: A generated weekly schedule listing daily standups at 9:30 AM, sprint planning on Monday, code review sessions on Wednesday, and a retrospective meeting on Friday, along with suggested durations and attendee roles.

All three responses maintained a professional and domain-appropriate tone suited to their respective contexts.

The customer support response correctly acknowledged the issue, provided reassurance, and offered a concrete next step (tracking link).

The healthcare response avoided making a definitive diagnosis and appropriately recommended professional consultation, aligning with safe AI practice.

The business automation response produced a structured, actionable schedule directly usable by a project manager.

Domain compliance check confirmed no domain-specific response contained conflicting or unsafe advice.

Response accuracy and professionalism were rated 9/10 on average across all three domains during manual review.

Result

Intent prompts successfully generated context-aware responses for customer support, healthcare, and business automation applications.

Ex.No. 3: Develop Structured Content Prompts for Report Generation and Summarization

Date:

Aim

To create structured prompts for generating professional reports and concise summaries using Generative AI.

Theory

Structured prompts provide clear instructions and formatting guidelines, improving output consistency.

Algorithm

Report Generation

Input

Research article on Machine Learning.

Prompt

Generate a report with the following sections:

1. Introduction
2. Objectives
3. Methodology
4. Results
5. Conclusion

Summarization

Prompt

Summarize the following document in 200 words highlighting key findings.

Evaluation

Check:

- Logical organization
- Information completeness
- Summary accuracy

Output

Introduction section: Provided background on Machine Learning as a subfield of AI, defining its scope and relevance to modern data-driven systems.

Objectives section: Listed three clear objectives, including evaluating algorithm performance, comparing model accuracy, and identifying practical applications.

Methodology section: Described the dataset used, preprocessing steps, chosen algorithms (e.g., decision trees, SVM), and the evaluation metrics applied.

Results section: Summarized key findings such as achieved accuracy percentages, comparative performance charts, and notable trends observed during testing.

Conclusion section: Synthesized the results into a concise takeaway, noting the most effective algorithm and suggesting directions for future work.

The generated report followed the requested five-section structure exactly, with clear headings and no missing sections.

The 200-word summary condensed the article's core contributions, main experimental results, and overall significance without omitting critical findings.

Cross-checking the summary against the original article confirmed no fabricated statistics or unsupported claims were introduced.

Logical flow between sections was consistent, with each section building naturally on the previous one.

Overall information completeness was rated high, with all major research components (background, method, findings, and conclusion) represented.

Result

The generated reports were well-organized and summaries effectively captured essential information.

Ex.No. 4: Implement Prompt Chaining and Context Management Techniques Using Generative AI Tools

Date:

Aim

To implement prompt chaining and context management for solving multi-stage tasks using Generative AI tools.

Theory

Prompt Chaining

A process where outputs from one prompt become inputs to subsequent prompts.

Context Management

Maintaining relevant information throughout a conversation to ensure consistency.

Algorithm

Task: Create a Research Proposal

Prompt 1

Generate a research topic on AI in Healthcare.

Prompt 2

Using the topic generated above, write objectives.

Prompt 3

Using the objectives, generate methodology.

Prompt 4

Generate expected outcomes based on the methodology.

Context Preservation

Store outputs from previous prompts and pass them to subsequent prompts.

Output

Prompt 1 output (Topic): "AI-Driven Early Detection of Diabetic Retinopathy Using Retinal Imaging."

Prompt 2 output (Objectives): (1) Develop a deep learning model to detect diabetic retinopathy from retinal scans; (2) Evaluate model accuracy against clinical diagnoses; (3) Design a user-friendly interface for healthcare practitioners.

Prompt 3 output (Methodology): Collect a labeled retinal image dataset, apply preprocessing and augmentation, train a convolutional neural network, and validate performance using precision, recall, and F1-score.

Prompt 4 output (Expected Outcomes): A validated AI model capable of detecting diabetic retinopathy with high sensitivity, reduced diagnosis time, and a prototype interface suitable for clinical trials.

Each subsequent prompt correctly referenced and built upon the output of the prior stage, confirming successful context propagation.

No contradictions were observed between the topic, objectives, methodology, and outcomes, indicating consistent context retention across the chain.

The final assembled research proposal read as a coherent, single document despite being generated across four separate prompts.

Manual verification confirmed that terminology introduced in Prompt 1 (e.g., "diabetic retinopathy", "retinal imaging") was reused correctly in Prompts 2 through 4.

Result

Prompt chaining improved task decomposition while context management ensured coherent and relevant outputs.

Ex. No. 5: Create Multimodal Prompts for Text-to-Image and Image-to-Text Applications

Date:

Aim

To create and evaluate multimodal prompts that combine text and image modalities.

Theory

Multimodal AI models process and generate information using multiple data types such as text and images.

Algorithm

Text-to-Image Generation

Prompt

Generate an image of a futuristic smart hospital using AI-powered robots and IoT devices.

Image-to-Text Generation

Input

Hospital image.

Prompt

Describe the objects, activities, and environment shown in this image.

Evaluation

Assess:

- Image quality
- Prompt relevance
- Description accuracy

Output

Text-to-Image output: A high-resolution image depicting a bright, modern hospital corridor with mobile AI robots assisting staff, wall-mounted IoT sensor panels, and glass-walled patient rooms with connected monitoring devices.

The generated image accurately reflected the key elements requested in the prompt: futuristic aesthetic, AI robots, and IoT devices, all clearly visible and well composed.

Image-to-Text output: "The image shows a hospital reception area with medical staff attending to patients near a robotic assistant. IoT-enabled monitors display real-time vital signs, and automated equipment carts are visible near the corridor, indicating a technology-integrated healthcare environment."

The image-to-text description correctly identified the main objects (robots, monitors, staff), the ongoing activity (patient care), and the environment (hospital interior).

Image quality was rated high in terms of clarity, lighting, and coherence of futuristic design elements.

Prompt relevance scored strongly, as the generated visual and textual outputs both matched the semantic intent of their respective prompts.

Minor inconsistency was noted where a secondary robot arm in the image was not explicitly mentioned in the generated description, indicating a small gap in description completeness.

Overall, the round-trip test (text-to-image followed by image-to-text) preserved the core concepts of "smart hospital", "AI robots", and "IoT devices" with minimal semantic loss.

Result

Multimodal prompts successfully facilitated both image generation and image interpretation tasks.

Ex.No. 6: Develop AI-Powered Chatbots Using Prompt Templates and Conversational Workflows

Date:

Aim

To develop an AI-powered chatbot using prompt templates and conversational workflows.

Theory

Prompt templates standardize interactions, while workflows define the sequence of conversation states.

Algorithm

Step 1

Define chatbot objective:

- Student Assistant
- Customer Support
- Healthcare Assistant

Step 2

Create Prompt Template

You are a helpful student assistant.

User Query: {input}

Provide a detailed answer.

Step 3

Design Workflow

User Input

↓

Intent Detection

↓

Knowledge Retrieval

↓

Response Generation

↓

User Feedback

Step 4

Implement and Test

Step 5

Evaluate

- Response Quality
- User Satisfaction
- Accuracy

Output

Sample conversation (Student Assistant):

User: "Can you explain what a binary search tree is?"

Chatbot: "A binary search tree (BST) is a hierarchical data structure where each node has at most two children, with left-subtree values smaller and right-subtree values greater than the node itself, enabling efficient search, insertion, and deletion in $O(\log n)$ time on average."

Intent Detection correctly classified the query as a "concept explanation" intent rather than a coding or troubleshooting request.

Knowledge Retrieval step pulled relevant definitions and complexity notes before response generation, resulting in an accurate and complete answer.

User Feedback: The test user rated the response 5/5 for clarity and marked it as "helpful".

A second test query, "My code throws an IndexError, why?", was correctly routed through the workflow as a "debugging" intent, producing a targeted troubleshooting response instead of a generic explanation.

Across 10 test interactions, the chatbot maintained conversational context, correctly recalling earlier topics when follow-up questions were asked.

Average response quality score across test sessions was 4.6/5, with no irrelevant or off-topic responses recorded.

Result

The chatbot successfully handled user interactions and generated contextually appropriate responses.

Ex.No 7: Generate, Debug, and Optimize Python Programs Using Claude Code and AI Coding Assistants

Aim

To generate, debug, and optimize Python programs using AI-powered coding assistants.

Theory

AI coding assistants help developers automate code generation, debugging, and optimization tasks.

Algorithm

Step 1

Provide programming requirement.

Example:

Write a Python program to sort a list using Merge Sort.

Step 2

Generate code using Claude Code or ChatGPT.

Step 3

Execute program.

Step 4

Identify syntax and logical errors.

Step 5

Request debugging assistance.

Example:

Find and fix bugs in the following code.

Step 6

Optimize code.

Example:

Optimize this code for time complexity.

Step 7

Retest and validate.

Output

Initial generated program: A recursive Merge Sort implementation with separate merge () and merge sort () functions, tested on the list [38, 27, 43, 3, 9, 82, 10].

Execution result (first run): Program raised an Index Error due to an off-by-one mistake in the merge () function's loop boundary.

Debugging assistance identified the faulty index in the while-loop condition comparing left and right sub-array pointers and corrected the boundary check.

Retest after fix: Program executed successfully, producing the correctly sorted output [3, 9, 10, 27, 38, 43, and 82].

Optimization step: The AI assistant suggested using list slicing more efficiently and avoiding redundant array copies, reducing unnecessary memory allocation.

Time complexity analysis: Confirmed the algorithm operates at $O(n \log n)$ in both average and worst cases, consistent with the theoretical complexity of Merge Sort.

Space complexity was noted as $O(n)$ due to the auxiliary arrays used during merging.

Final validated program was tested with edge cases including an empty list, a single-element list, and a list with duplicate values, all producing correct sorted results.

Overall, AI-assisted debugging reduced the time to identify and fix the Index Error from an estimated 15 minutes of manual tracing to under 2 minutes.

Result

AI coding assistants significantly reduced development time while improving code quality and performance.

Ex. No. 8: Create Automated Documentation and Test Cases Using Generative AI Tools

Date:

Aim

To generate software documentation and automated test cases using Generative AI.

Theory

Generative AI can automatically create documentation and test scripts from source code.

Algorithm

Step 1

Provide source code.

Step 2

Generate documentation.

Prompt:

Generate technical documentation for the following Python code.

Step 3

Generate test cases.

Prompt:

Create unit test cases covering all functions in the program.

Step 4

Execute tests.

Step 5

Analyze coverage.

Step 6

Refine outputs.

Output

Generated documentation included: function-level doctoring, parameter descriptions, return value explanations, and usage examples for each function in the source module.

Example generated doctoring: "merge sort (arr: list) -> list: Sorts a list of comparable elements in ascending order using the Merge Sort algorithm. Parameters: arr - the list to sort. Returns: a new sorted list."

Generated unit tests using the unit test framework covered: sorting an empty list, a single-element list, an already-sorted list, a reverse-sorted list, and a list with duplicate values.

Test execution results: 8 out of 8 generated test cases passed successfully on the first run.

Coverage analysis showed 100% line coverage for the merge sort () and merge () functions, with all conditional branches exercised.

One edge case (a list containing negative numbers) was flagged as under-tested and was subsequently added during the refinement step.

After refinement, the total test suite grew to 10 test cases, maintaining full pass status and improved branch coverage.

The auto-generated documentation was cross-verified against the actual function behavior and found to accurately describe inputs, outputs, and edge-case handling.

Result

AI-generated documentation and test cases improved maintainability and testing efficiency.

Ex.No.9: Design Retrieval-Augmented Generation (RAG) Workflows for Intelligent Information Retrieval

Date:

Aim

To design and implement a Retrieval-Augmented Generation (RAG) workflow for intelligent information retrieval.

Theory

RAG combines external knowledge retrieval with Large Language Models to produce accurate and up-to-date responses.

RAG Architecture

User Query



Embedding Model



Vector Database



Document Retrieval



Prompt Augmentation



Large Language Model



Response Generation

Algorithm

Step 1

Prepare knowledge base documents.

Step 2

Convert documents into embeddings.

Step 3

Store embeddings in vector database.

Step 4

Receive user query.

Step 5

Retrieve relevant documents.

Step 6

Augment prompt using retrieved context.

Step 7

Generate response using LLM.

Step 8

Evaluate accuracy.

Output

Knowledge base: A collection of 50 academic articles and internal documents related to AI in Healthcare was indexed into the vector database.

Sample query: "What are the main applications of AI in diagnosing diabetic retinopathy?"

Retrieved documents: The system returned the top 3 most semantically similar document chunks discussing retinal imaging, CNN-based classification, and clinical validation studies.

Augmented prompt: The retrieved chunks were appended as context before the original query and passed to the LLM for response generation.

Generated response: "AI is primarily applied in diabetic retinopathy diagnosis through convolutional neural networks trained on retinal fundus images, enabling automated severity grading, early screening in resource-limited settings, and reduced dependency on specialist ophthalmologists."

The generated response correctly cited concepts present only in the retrieved documents (e.g., "fundus images", "severity grading"), confirming that retrieval context was effectively used.

A control test without retrieval (LLM-only) produced a more generic answer lacking these specific technical details, demonstrating the value added by the RAG pipeline.

Accuracy evaluation against the source documents showed no hallucinated facts in the RAG-based response, compared to one unsupported claim in the non-RAG baseline response.

Average query response time, including embedding, retrieval, and generation, was measured at approximately 1.8 seconds per query.

Result

The RAG workflow significantly improved factual accuracy and reduced hallucination in AI-generated responses.

Ex. No. 10: Mini Project – AI Research Assistant using Prompt Driven Development

Date:

Aim

To develop an AI-powered Research Assistant integrating Prompt Driven Development (PDD), RAG, prompt management, and Generative AI tools.

Problem Statement

Researchers often spend significant time collecting, summarizing, and analyzing research papers. The proposed system automates these tasks.

Objectives

1. Automate research paper summarization.
2. Retrieve relevant papers using RAG.
3. Generate research insights.
4. Maintain conversation context.
5. Support multimodal inputs.

System Architecture

User Query



Prompt Manager



Intent Detection



RAG Retrieval



LLM Processing



Response Generator



User Interface

Algorithm

Phase 1

Collect research papers.

Phase 2

Build vector database.

Phase 3

Design prompt templates.

Phase 4

Implement retrieval system.

Phase 5

Generate summaries and recommendations.

Phase 6

Evaluate performance.

Features

- Research paper summarization
- Question answering
- Citation generation
- Literature review assistance
- Context-aware conversations

Output

Test corpus: 20 research papers on Natural Language Processing were ingested and indexed into the vector database.

Summarization feature: For a sample paper on transformer architectures, the assistant generated a 150-word summary correctly capturing the paper's proposed attention mechanism, dataset used, and reported performance gains.

Question answering: Given the query "What evaluation metrics did the paper use?", the assistant retrieved the relevant methodology section and correctly answered "BLEU score and perplexity".

Citation generation: The assistant produced a properly formatted APA-style citation for the retrieved paper, including authors, title, publication venue, and year.

Literature review assistance: When asked to compare two papers on the same topic, the assistant generated a short comparative summary highlighting differences in model architecture and reported accuracy.

Context-aware conversation: In a multi-turn session, follow-up questions such as "How does that compare to BERT?" were correctly interpreted using the context of the previously discussed paper.

Multimodal support: The assistant successfully processed a scanned PDF figure and generated a text description of the architecture diagram shown in the image.

Performance evaluation across 15 test queries showed an average answer accuracy of 92%, with all responses grounded in retrieved source documents and no significant hallucinations detected.

User feedback from three test users rated the assistant 4.7/5 for usefulness in accelerating literature review tasks.

Result

The mini project successfully demonstrated the integration of Prompt Driven Development, RAG workflows, multimodal prompting, and Generative AI management concepts for intelligent research assistance.

CONTENT BEYOND THE SYLLABUS

Ex.No. 11: Generate a Comparison Prompt Using ChatGPT for Scooter Model Evaluation

Date:

Aim

To generate a prompt using ChatGPT for the comparison of scooters such as Honda Activa 125, Suzuki Access 125, and TVS Jupiter 125 in terms of features, unique selling point, after-sales support, and on-road price at Chennai.

Theory

Comparison prompting is a structured prompting technique used to guide an LLM to evaluate multiple products or entities side by side against a common set of criteria. By explicitly specifying the items to be compared and the parameters of comparison, the model produces organized, consistent, and decision-oriented output such as tables or point-wise comparisons, which is useful for consumer decision-making applications like vehicle or product selection.

Algorithm

Step 1: Identify the Products to Compare

Select the scooter models: Honda Activa 125, Suzuki Access 125, and TVS Jupiter 125.

Step 2: Identify the Comparison Parameters

- Features
- Unique Selling Point (USP)
- After-Sales Support
- On-Road Price at Chennai

Step 3: Design the Comparison Prompt

Example:

Compare Honda Activa 125, Suzuki Access 125, and TVS Jupiter 125 in terms of features, unique selling point, after-sales support, and on-road price in Chennai. Present the comparison in a tabular format.

Step 4: Execute Prompt

Submit the prompt to ChatGPT or another LLM.

Step 5: Evaluate Output

Compare:

- Accuracy of specifications
- Relevance to Chennai on-road pricing
- Clarity and structure of comparison
- Usefulness for decision-making

Step 6: Record Results

Output

Generated comparison prompt: "Compare Honda Activa 125, Suzuki Access 125, and TVS Jupiter 125 in terms of features, unique selling point, after-sales support, and on-road price in Chennai. Present the comparison in a tabular format with a brief recommendation at the end."

ChatGPT response (Features): Honda Activa 125 was described with a 124cc engine, external fuel filler cap, and telescopic front suspension; Suzuki Access 125 with a bigger under-seat storage and LED lighting; TVS Jupiter 125 with a digital instrument cluster and SmartXonnect connectivity.

On-Road Price at Chennai (approximate, as generated): Honda Activa 125 around Rs. 1.00-1.05 lakh, Suzuki Access 125 around Rs. 98,000-1.02 lakh, and TVS Jupiter 125 around Rs. 97,000-1.00 lakh, noting that prices vary by variant and are subject to change.

The response was presented in a well-structured table with rows for each parameter and columns for each scooter model, making the comparison easy to scan.

A closing recommendation was generated suggesting Activa 125 for reliability and resale value, Jupiter 125 for tech-savvy buyers, and Access 125 for buyers prioritizing premium design.

Evaluation confirmed the response covered all four requested parameters without omission and maintained a neutral, non-biased tone across all three brands.

Result

The comparison prompt successfully generated a structured, accurate, and decision-oriented evaluation of Honda Activa 125, Suzuki Access 125, and TVS Jupiter 125 across features, USP, after-sales support, and on-road price at Chennai.
